

Unit 5: Implementation and Maintenance (4 Hrs.)

5.1.System Implementation:

Introduction, System Implementation, Software Application Testing, Installation, Documenting the System, Training and Supporting Users, Organizational Issues in Systems Implementation

5.2.Maintaining Information Systems: Introduction, Maintaining Information Systems, Conducting Systems Maintenance

- **Introduction:**

- **Implementation** is a process of ensuring that the information system is operational. It involves – Constructing a new system from scratch.
- **Systems implementation** is the process of: defining how the information **system** should be built (i.e., physical **system** design), ensuring that the information **system** is operational and used, ensuring that the information **system** meets quality standard (i.e., quality assurance).
- Implementation is a procedure of ensuring that the information system is operational. It includes –
 - Constructing a new system from scratch
 - Constructing a new system from the present one.
- Implementation allows the users to take over its operation to be used and evaluation. It involves training the users to handle the system and plan for a smooth conversion.

- **The purpose of implementation**

- is to build a properly working system (To convert final physical system specifications into working and reliable software)
 - install it in the organization,
 - replace old systems and work methods,
 - finalize system and user documentation (To document work that has been done)
 - train users (To provide help for current and future users), and
 - prepare support systems to assist users
- Implementation also involves closedown of the project, including evaluating personnel, reassigning staff, assessing the success of the project, and turning all resources over to those who will support and maintain the system

- **The purpose of maintenance** is to fix and enhance the system to respond to problems and changing business conditions.
- Maintenance includes activities from all systems development phases. Maintenance also involves responding to requests to change the system, transforming requests into changes, designing the changes, and implementing them.

Six major activities of implementation:

- Coding
 - Testing
 - Installation
 - Documentation
 - Training
 - Support
-
- The purpose of these steps is to convert the physical system specifications into working and reliable software and hardware, document the work that has been done, and provide help for current and future users and caretakers of the system.
 - Coding and testing may have already been completed by this point if Agile Methodologies have been followed.
 - Using a plan-driven methodology, coding and testing are often done by other project team members besides analysts, although analysts may do some programming.
 - In any case, analysts are responsible for ensuring that all of these various activities are properly planned and execute

System Implementation

- **Coding:**

- Coding is the process where, physical design specifications created by the analysis team are turned into working computer code by program team.
- *Coding Deliverables:* Code, Program Documentation

- **Testing:**

- Testing is the process of finding and fixing bugs and errors. Tests are performed using various strategies. Testing performed in parallel with coding.
- *Testing Deliverables:* Test Scenarios (test plan) and test data, Results of programs and system testing

- **Installation:**

- It is the process during which the current system is replaced by new system.
- *Installation Deliverables:* User guides, User training plan, Installation and conversion plan

System Implementation

- **Documentation:**

- It is the process of converting software specification with the hard copy format it acts as references maintained for user . Two audiences for final documentation
- Information systems personnel (System Builders, internal users) who will maintain the system throughout its productive life.
- People who will use the system (System Users) as part of their daily lives.
- *Deliverables: System Documentation, User Documentation*

- **User Training:**

- It is the process of providing necessary skills and knowledge to user. The training can be Application specific or General for operating system and off the shelf software.
- *Deliverables: Classes, Tutorials, User training modules, training materials, and computer based training aids*

- **User Support Plan**

- It is the process of building of mechanism which acts as guidelines for the user in case of any problem arises
- *Deliverables: Help desk, online help, Bulletin boards and other support mechanisms*

Table 15-1 Deliverables for Coding, Testing, and Installation

1. Coding	3. Installation
a. Code	a. User guides
b. Program documentation	b. User training plan
2. Testing	c. Installation and conversion plan
a. Test scenarios (test plan) and test data	i. Software and hardware installation schedule
b. Results of program and system testing	ii. Data conversion plan
	iii. Site and facility remodeling plan

Table 15-2 Deliverables for Documenting the System, Training, and Supporting Users

1. Documentation	3. User training modules
a. System documentation	a. Training materials
b. User documentation	b. Computer-based training aids
2. User training plan	4. User support plan
a. Classes	a. Help desk
b. Tutorials	b. Online help
	c. Bulletin boards and other support mechanisms

Software Testing

- process used to help identify the correctness, completeness, security and quality of developed computer system.
- Testing is a process of technical investigation performed on behalf on stakeholders that is intended to reveal quality related information about the product with respect to the context in which it is integrated to operate.
- This includes the process of executing a program or application with the intend of finding error.
- No program or system design is perfect , communication between the user and the designer is not always short .

Types of Testing

[A] Based on Design of Test Case

- Black Box Testing-
 - Functional
 - behavioral Testing
 - Cause effect testing
 - Equivalence class Testing
 - State Based Testing
- White box Testing
 - Structural Testing
 - Basic path testing
 - Loop Testing

[B] Based on the level of Testing

1. Unit Testing
2. Integration Testing/Interface/Component Testing
 - Big Bang
 - Incremental Approach
 - Top Down Testing ,
 - Bottom Up Testing
 - Sandwich

3. System Testing

- Stress testing
- Performance Testing
- Security Testing
- Usability Testing
- Documentation Testing
- Response Testing
- Recovery Testing

4. User Acceptance Testing

- Alpha Testing
- Beta Testing

[C] Based on the location Of testing

- Alpha Testing
- Beta Testing
- Regression Testing

- **Software Application Testing**
- Software testing can be stated as the process of verifying and validating that a software or application is bug free, meets the technical requirements as guided by its design and development and meets the user requirements effectively and efficiently with handling all the exceptional and boundary cases.
- Software testing is method of assessing the functionality of a software program.
- The process of software testing aims not only at finding faults in the existing software but also at finding measures to improve the software in terms of efficiency, accuracy and usability.
- It mainly aims at measuring specification, functionality and performance of a software program or application.
- **A master test plan is developed during the analysis phase.**
- **During the design phase, unit, system and integration test plans are developed.**
- **The actual testing is done during implementation.**
- **Test plans provide improved communication among all parties involved in testing.**

Table 15-3 Table of Contents of a Master Test Plan

1. Introduction	4. Procedure Control
a. Description of system to be tested	a. Test initiation
b. Objectives of the test plan	b. Test execution
c. Method of testing	c. Test failure
d. Supporting documents	d. Access/change control
2. Overall Plan	e. Document control
a. Milestones, schedule, and locations	5. Test-Specific or Component-Specific Test Plans
b. Test materials	a. Objectives
1. Test plans	b. Software description
2. Test cases	c. Method
3. Test scenarios	d. Milestones, schedule, progression, and locations
4. Test log	e. Requirements
c. Criteria for passing tests	f. Criteria for passing tests
3. Testing Requirements	g. Resulting test materials
a. Hardware	h. Execution control
b. Software	i. Attachments
c. Personnel	

(Source: Adapted from Mosley, 1993.)

Test Classification

- Manual vs. Automated
- Static (syntax only) vs. Dynamic (execution)

Table 15-4 A Categorization of Test Types

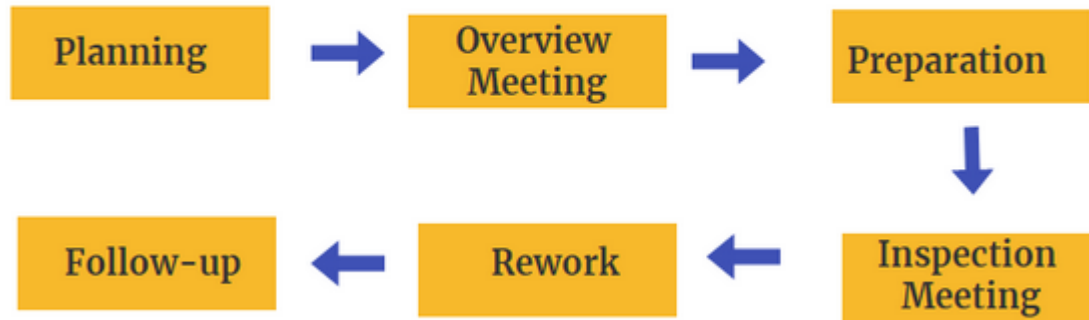
	<i>Manual</i>	<i>Automated</i>
<i>Static</i>	Inspections	Syntax checking
<i>Dynamic</i>	Walkthroughs Desk checking	Unit test Integration test System test

(Source: Adapted from Mosley, 1993.)

Manual Testing Techniques: Inspection, Walkthrough and desk checking

Software inspection is a process of finding defects and deficiencies in a developed software product and ensuring that it fulfills the client's requirements.

PROCESS:



BENEFITS:

- Finds defects or faults.
- Improves quality, such as coding style, best practices, etc.
- Assists with debugging.
- Educates and trains developers.
- Communication is improved.
- Collects data for metrics.



Participants and Roles:

During software inspection, all the participants have a set role. However, more than one role can be assigned to the same person, in which case the team size may vary from one inspection to another. The roles of these participants are:

Author

Programmer or designer who is responsible for producing the program/document that is being inspected.

Leader of the inspection who is responsible for planning the inspection & coordinating it.

Moderator

Reader

Present the code at an inspection meeting, where they read the document one by one.

A participant who is responsible for documenting the defects found during the inspection process.

Recorder/ Scribe

Inspector

Inspectors are allotted the task of examining the work product to identify possible defects.

• Inspection

- A testing technique in which participants examine program code for predictable language-specific errors that is participants manually examine code for occurrences of well-known errors.
- Syntax, grammar, and some other routine errors can be checked by automated inspection software, so manual inspection checks are used for more subtle (small) errors.
- Each programming language lends itself to certain types of errors that programmers make when coding, and these common errors are well known and documented.
- Exactly what the code does is not investigated in an inspection.
- It has been estimated that code inspections detect from 60 to 90 percent of all software defects as well as provide programmers with feedback that enables them to avoid making the same types of errors in future work (Fagan, 1986).

- **Walkthrough**

- A peer group review of any product created during the systems development process; also called a structured walkthrough.
- A walkthrough guides reviewers through the review of a work product.
- The author of the work product presents the design and ensures that the attendees understand its design.
- Walkthroughs allow people with less experience to review a work product
- Users of the work product are often invited to walkthroughs
- Prior to the walkthrough, the author should distribute presentation materials to attendees.
- During the walkthrough, the author should solicit feedback from the audience.
- After the walkthrough, the author should follow up with attendees who have helped out by giving comments .

- **Desk Checking**

- A testing technique in which the program code is sequentially executed manually by the reviewer.
- It is informal process in which the programmer or someone else who understands the logic of the program works through the code with a paper and pencil.
- The programmer executes each instruction, using test cases that may or may not be written down.
- In one sense, the reviewer acts as the computer, mentally checking each step and its results for the entire set of computer instructions.

S.No.	Inspection	Walkthrough
1.	It is formal.	It is informal.
2.	Initiated by project team.	Initiated by author.
3.	A group of relevant persons from different departments participate in the inspection.	Usually team members of the same project take participation in the walkthrough. Author himself acts walkthrough leader.
4.	Checklist is used to find faults.	No checklist is used in the walkthrough.
5.	Inspection processes includes overview, preparation, inspection, and rework and follow up.	Walkthrough process includes overview, little or no preparation, little or no preparation examination (actual walkthrough meeting), and rework and follow up.
6.	Formalized procedure in each step.	No formalized procedure in the steps.
7.	Inspection takes longer time as list of items in checklist is tracked to completion.	Shorter time is spent on walkthrough as there is no formal checklist used to evaluate program.
8.	Planned meeting with the fixed roles assigned to all the members involved.	Unplanned
9.	Reader reads product code. Everyone inspects it and comes up with detects.	Author reads product code and his teammate comes up with the defects or suggestions.
10.	Recorder records the defects.	Author make a note of defects and suggestions offered by teammate.
11.	Moderator has a role that moderator making sure that the discussions proceed on the productive lines.	Informal, so there is no moderator.

Automated Testing Techniques

- **Syntax Checking**

- The compiler is run against the source code to identify syntax errors.
- Syntax Testing, a black box testing technique, involves testing the System inputs and it is usually automated because syntax testing produces a large number of tests.
- Internal and external inputs have to conform the below formats:
 - Format of the input data from users.
 - File formats.
 - Database schemas.

- **Syntax Testing - Steps:**

- Identify the target language or format.
- Define the syntax of the language.
- Validate and Debug the syntax.

- **Syntax Testing - Limitations:**

- Sometimes it is easy to forget the normal cases.
- Syntax testing needs driver program to be built that automatically sequences through a set of test cases usually stored as data.

Automated Testing Techniques

- **Unit Testing**

- Each module is tested alone in an attempt to discover any errors in its code, also called module testing.
- Unit testing, a testing technique using which individual modules are tested to determine if there are any issues by the developer himself. It is concerned with functional correctness of the standalone modules.
- The main aim is to isolate each unit of the system to identify, analyze and fix the defects.
- **Unit Testing - Advantages:**
 - Reduces Defects in the Newly developed features or reduces bugs when changing the existing functionality.
 - Reduces Cost of Testing as defects are captured in very early phase.
 - Improves design and allows better refactoring of code.
 - Unit Tests, when integrated with build gives the quality of the build as well.

• Integration Testing

- The process of bringing together all of the modules that a program comprises for testing purposes. Modules are typically integrated in a top-down, incremental fashion.
- **INTEGRATION TESTING** is defined as a type of testing where software modules are integrated logically and tested as a group.
- A typical software project consists of multiple software modules, coded by different programmers.
- The purpose of this level of testing is to expose defects in the interaction between these software modules when they are integrated
- Upon completion of unit testing, the units or modules are to be integrated which gives rise to integration testing.
- The purpose of integration testing is to verify the functional, performance, and reliability between the modules that are integrated.
- The goal of integration testing is to check the correctness of communication among all the modules.

- **Why Integration Testing?**

- Integration Testing becomes necessary to verify the software modules work in unity.
- At the time of module development, there are wide chances of change in requirements by the clients. These new requirements may not be unit tested and hence system integration Testing becomes necessary.
- Interfaces of the software modules with the database could be erroneous
- External Hardware interfaces, if any, could be erroneous
- Inadequate exception handling could cause issues.

- **Integration Strategies:**

- Big-Bang Integration
- Top Down Integration
- Bottom Up Integration
- Hybrid Integration

Automated Testing Techniques (cont.)

- **System Testing**

- The bringing together of all the programs that a system comprises for testing purposes. Programs are typically integrated in a top-down, incremental fashion.
- **SYSTEM TESTING** is a level of testing that validates the complete and fully integrated software product. The purpose of a system test is to evaluate the end-to-end system specifications.
- System Testing includes testing of a fully integrated software system.
- System Testing is actually a series of different tests whose sole purpose is to exercise the full computer-based system.
- System testing tests the design and behavior of the system and also the expectations of the customer.
- System Testing is basically performed by a testing team that is independent of the development team that helps to test the quality of the system impartial. It has both functional and non-functional testing.

Types of System Testing



Regression Testing

Regression testing is performed under system testing to confirm and identify that if there's any defect in the system due to modification in any other part of the system. It makes sure, any changes done during the development process have not introduced a new defect and also gives assurance; old defects will not exist on the addition of new software over the time.

Load Testing

Load testing is performed under system testing to clarify whether the system can work under real-time loads or not.

Functional Testing

Functional testing of a system is performed to find if there's any missing function in the system.

Types of System Testing



Recovery Testing

Recovery testing of a system is performed under system testing to confirm reliability, trustworthiness, accountability of the system and all are lying on recouping skills of the system. It should be able to recover from all the possible system crashes successfully.

Migration Testing

Migration testing is performed to ensure that if the system needs to be modified in new infrastructure so it should be modified without any issue.

Usability Testing

The purpose of this testing to make sure that the system is well familiar with the user and it meets its objective for what it supposed to do.

Software and Hardware Testing

This testing of the system intends to check hardware and software compatibility. The hardware configuration must be compatible with the software to run it without any issue. Compatibility provides flexibility by providing interactions between hardware and software.

Automated Testing Techniques (cont.)

- Stub Testing

- A technique used in testing, especially where modules are written and tested in a top-down fashion, where a few lines of code are used to substitute for subordinate modules
- Stubs are two or three lines of code written by a programmer to stand in for the missing modules.
- During testing, the coordinating module calls the stub instead of the subordinate module.
- The stub accepts control and then returns it to the coordinating module.

User Acceptance Techniques

- Actual users test a completed information system.
- End result is the users' final acceptance of the system.
- Alpha testing: use simulated data
- Beta testing: use real data in real user environment
- **Types of Alpha Testing**
 - Recovery testing
 - Forces software (or environment) to fail in order to verify that recovery is properly performed
 - Security testing
 - Verifies that protection mechanisms built into the system will protect it from improper penetration
 - Stress testing
 - Tries to break the system
 - Performance testing
 - Determines how the system performs on the range of possible environments in which it may be used
- **Use of Acceptance Testing:**
 - To find the defects missed during the functional testing phase.
 - How well the product is developed.
 - A product is what actually the customers need.
 - Feedbacks help in improving the product performance and user experience.
 - Minimize or eliminate the issues arising from the production.

Alpha testing vs Beta Testing

Sr. No.	Alpha Testing	Beta Testing
1.	Alpha testing performed by a team of highly skilled testers who are usually the internal employee of the organization.	Beta testing performed by clients or end-users in a real-time environment, who is not an employee of the organization.
2.	Alpha testing performed at the developer's site; it always needs a testing environment or lab environment.	Beta testing doesn't need any lab environment or the testing environment; it is performed at a client's location or end-user of the product.
3.	Reliability or security testing not performed in-depth in alpha testing.	Reliability, security, and robustness checked during beta testing.
4.	Alpha testing involves both white box and black-box techniques.	Beta testing uses only black-box testing.
5.	Long execution cycles maybe require for alpha testing.	Only a few weeks are required for the execution of beta testing.
6.	Critical issues or fixes can be identified by developers immediately in alpha testing.	Most of the issues or feedback is collecting from the beta testing will be implemented for the future versions of the product.
7.	Alpha testing performed before the launch of the product into the market.	At the time of software product marketing.
8.	Alpha testing focuses on the product's quality before going to beta testing.	Beta testing concentrates on the quality of the product, but gathers users input on the product and ensures that the product is ready for real-time users.
9.	Alpha testing performed nearly the end of the software development.	Beta testing is a final test before shipping a product to the customers.
10.	Alpha testing is conducting in the presence of developers and the absence of end-users.	Beta testing reversed of alpha testing.

White vs Black Box Testing

- **Black Box Testing:**

- It is concerned with input and output domain.
- It **focuses on functional requirements** to find out errors like incorrect or mission function, errors on data structures and performance.
- Functional/Behaviour Test

- **White Box Testing:**

- It is to ensure that a **logic implemented** in program modules are correct.
- This test executes **all the loops and conditions** within the operational boundaries.
- It is performed early in testing process and testing team must have a **good knowledge of testing process**.
- Structural Testing

Blackbox Testing/Cause Effect testing/	Whitebox Testing
the internal structure or the program or the code is hidden and nothing is known about it.	the tester has knowledge about the internal structure or the code or the program of the software.
mostly done by software testers.	mostly done by software developers.
No knowledge of implementation is needed.	Knowledge of implementation is required.
referred as outer or external software testing.	the inner or the internal software testing.
functional test of the software.	structural test of the software.
can be initiated on the basis of requirement specifications document.	is started after detail design document.
No knowledge of programming is required.	mandatory to have knowledge of programming.
behavior testing of the software.	logic testing of the software.
applicable to the higher levels of testing of software.	It is generally applicable to the lower levels of software testing.
It is also called closed testing.	It is also called as clear box testing.
It is least time consuming.	It is most time consuming.
It is not suitable or preferred for algorithm testing.	It is suitable for algorithm testing.
Can be done by trial and error ways and methods.	Data domains along with inner or internal boundaries can be better tested.
Example: search something on google by using keywords	Example: by input to check and verify loops
Types of Black Box Testing: A. Functional Testing B. Non-functional testing C. Regression Testing	Types of White Box Testing: A. Path Testing B. Loop Testing C. Condition testing

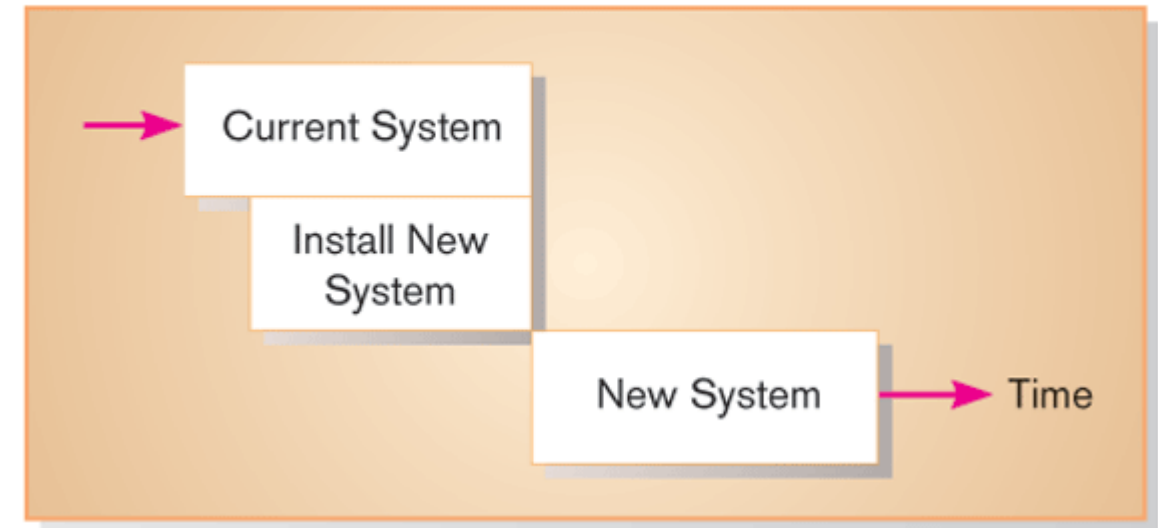
Installation

- The organizational process of changing over from the current information system to a new one
- Installation is the process of moving from the current system to the new or enhanced system.
- This is also refer to conversion activity; converting an old system to a new system.
- At this stage, all users must give up their reliance on the current system and begin to rely on a new system.
- There are four approaches of installation; direct, parallel, single-location and phased installation.
- Four installation strategies:
 - Direct Installation
 - Parallel Installation
 - Single-location installation
 - Phased Installation

1.Direct Installation:

- Involves the Old system being completely dropped and the new system being completed installed at the same time
- Need to make sure the new system is completely functional and operational
- This conversion method is only used when it is not feasible to operate separate systems at the same time
- Any data and training must be done before the installation
- This approach is a high risk approach because there may still be major problems that won't be uncovered until the system has been in operation.
- An errors resulting from the new system will have a direct impact on the users and how they do their jobs is depends on the new system.
- If the system fails to be used on that specific date, delay may occur until the errors are solved.
- This approach may be necessary when for instance, there is a business policy becomes effective on a specific date and the system can't be implemented before the date.
- If the organization plan to use this approach, it's important to make sure that everything is success with the system, so that less risky.
- Although it's known as risky approach, but it may reduce the cost of installation.

Figure 15-6a Comparison of installation strategies - Direct installation

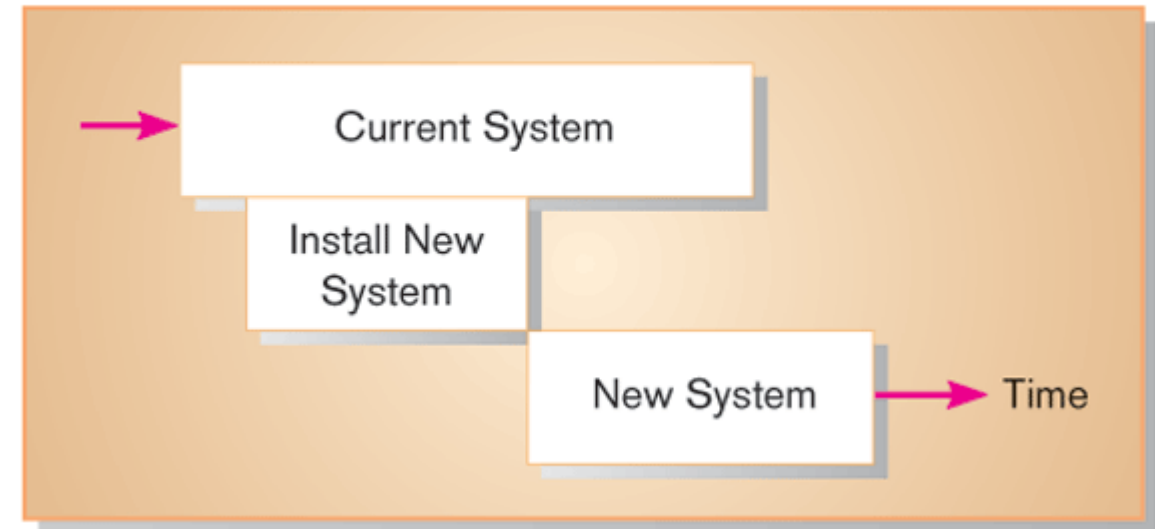


Turning off
the old
system when
the new one
is turned on

2.Parallel Installation:

- Involves the operating of both system being run for a period of time
- This allows problems with the new system to be found without loss of data.
- Once the new system is fully functional and operational the old system can stop being used.
- A consequence of using this is that old versions must continued being supported until the conversion.
- This approach is known as riskless approach because all major problems have been solved before the old system is turned off.
- This approach is an opposite approach with direct installation.
- Although it's riskless, but it required high costs since the organization needs to run two systems at the same time.
- Running two similar systems at one time can be very expensive not only in paying the employee salaries, but also in terms of maintenance.
- Sometimes, users are also confused since they must deal with two systems.

Figure 15-6b Comparison of installation strategies - Parallel installation

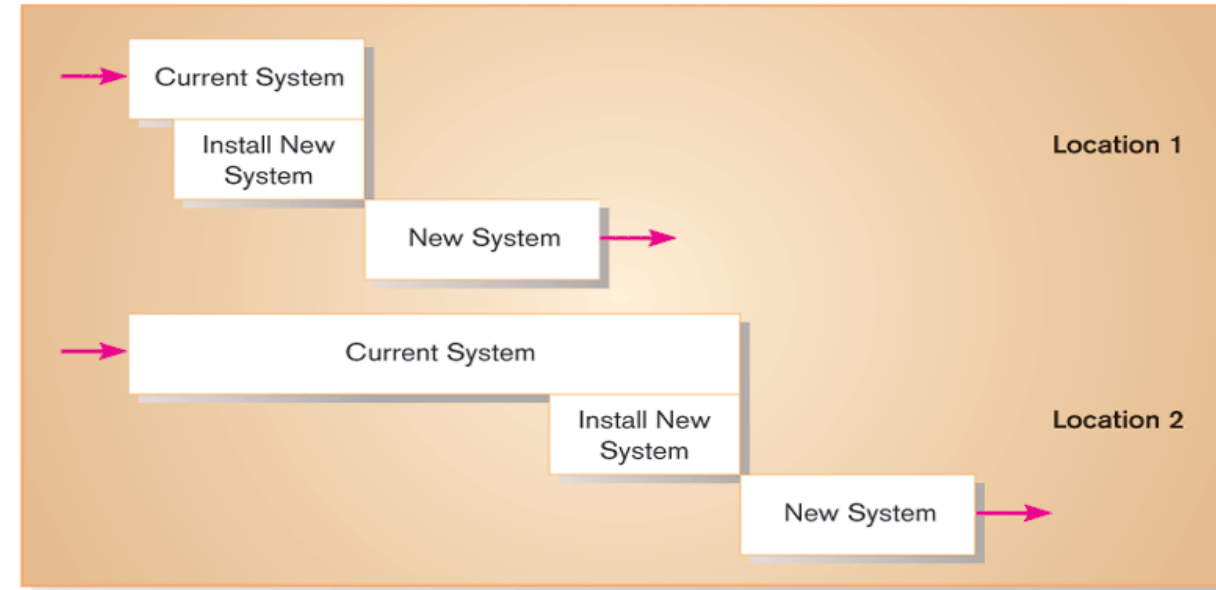


Running the old and new information systems at the same time until management decides the old system can be turned off

3.Single Location(Pilot)Installation:

- The new system is installed for a small number of users.
- Users learn, use and evaluate the new system.
- Once the system is seen as satisfactory the new system is installed and used by all.
- Allows users to become experts and teachers of the new system
- Pilot conversion also allows for testing the product in a operational setting
- The one site installation is either use direct installation or parallel installation.
- It's a middle-of-the-road approach compared with direct and parallel installation.
- Single location can be referring as branch office, or one department.
- As soon as the location approved the system, it will be install to the other location.
- At this stage, the installation at other site can be direct installation since all the major errors have been fixed at the first test location.
- The advantage of using this strategy is it can minimize the potential damage and costs because the successful is determined at the first site.
- But the disadvantage is, it may take a longer period to install the whole system.

Figure 15-6c Comparison of installation strategies - Single-location installation (with direct installation at each location)

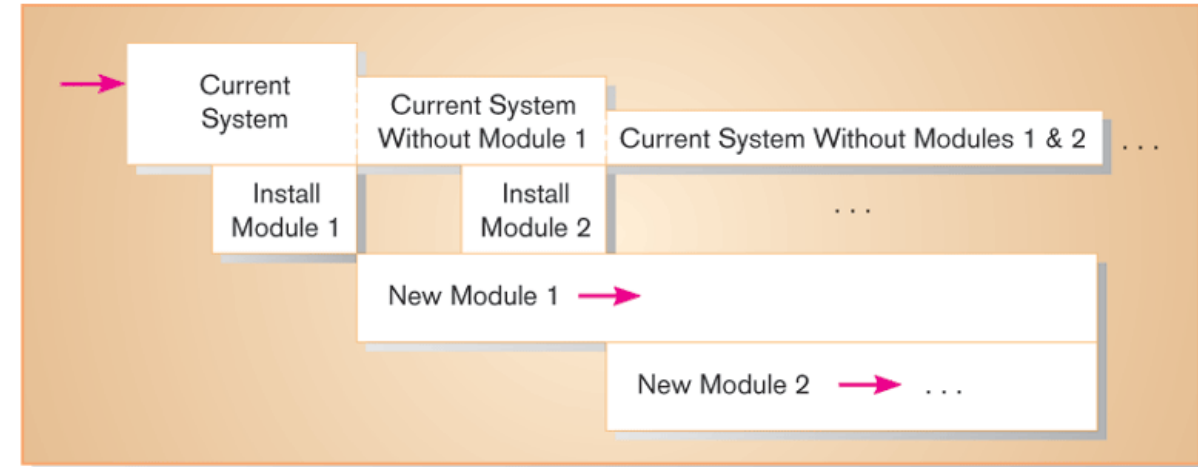


Trying out an information system at one site, then deciding if and how the new system should be deployed throughout the organization

4. Phased Installation:

- The phased method of installation from an old system to a new one through a gradual introduction of the new and discarding of the old
- This is usually done by introducing new parts of a system while removing/replacing old ones.
- Often used when the product as a whole is still under development.
- Overtime the entire system will be converted from old to new.
- It's an incremental approach where a new system is brought by module.
- Different modules of the new and old system are used together until the whole system is installed.
- Phased installation, like single-location installation is to minimize the risk exists from the installation.
- A phased installation is similar to bringing out a sequence of releases of the system.
- Thus, phased installation requires careful version control, repeated conversion at each phase and long period of change.
- It may cause the frustration and confusion among the users.

Figure 15-6d Comparison of installation strategies - Phased installation



Changing from the old information system to the new one incrementally, starting with one or a few functional components and then gradually extending the installation to cover the whole new system

Planning Installation

- Considerations
 - Data conversion
 - Error correction
 - Loading from current system
 - Planned system shutdown
 - Business cycle of organization

Documenting the System

- Every system development is unique and need unique documentation. There are two basic types of documentation; system documentation and user documentation.
- **Two audiences for documentation**
 - The information systems personnel who will maintain the system throughout its productive life
 - The people who will use the system as part of their daily lives
- **Deliverables**
 - Documentation
 - System documentation
 - User documentation
 - User training plan
 - Classes
 - Tutorials
 - User training modules
 - Training materials
 - Computer-based training aids
 - User support plan
 - Help desk
 - On-line help
 - Bulletin boards and other support mechanisms

Documenting the System

- **System documentation**

- Detailed information about a system's design specifications, its internal workings and its functionality
- Intended audience: maintenance programmers
- Can be divided into internal and External documentation.
- **Internal documentation:** embedded in the program source code or generated at compile time
- **External documentation:**
 - External documentation is a system documentation that includes the outcome of structured diagramming techniques such as data flow and entity-relationship diagrams.
 - Although it don't cover the code itself, but it can provide useful information to the primary users of system documentation-maintenance programmer

Documenting the System (cont.)

- User Documentation

- User documentation is intended for system users.
- User documentation consists of written and visual information about the system, such as what its function, how its function and how to use it.
- There are few types of user documentations;
 - reference guide - consist of exhaustive list of system functions and commands
 - quick reference guide - provide essential information about operating a system in a short, concise format
 - user's guide - provide information on how users can use a computer system to perform specific tasks.

Table 15-6

Outline of a Generic User's Guide

Preface

1. Introduction

1.1 Configurations

1.2 Function flow

2. User interface

2.1 Display screens

2.2 Command types

3. Getting started

3.1 Login

3.2 Logout

3.3 Save

3.4 Error recovery

3.n [Basic procedure name]

n. [Task name]

Appendix A—Error Messages

([Appendix])

Glossary

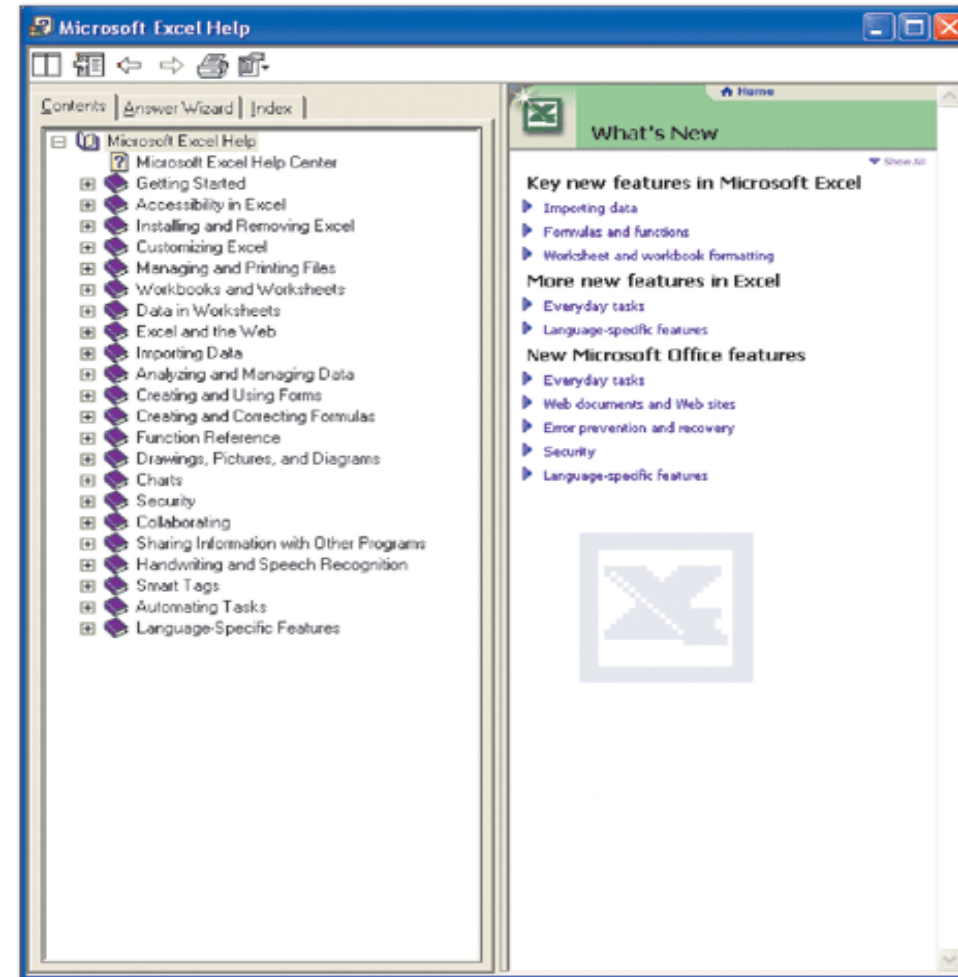
Terms

Acronyms

Index

(Source: Adapted from Bell and Evans, 1989.)

Figure 15-8 Contents of the help facility in Microsoft ExcelTM



User documentation is typically in the form of online help

Training Information Systems Users

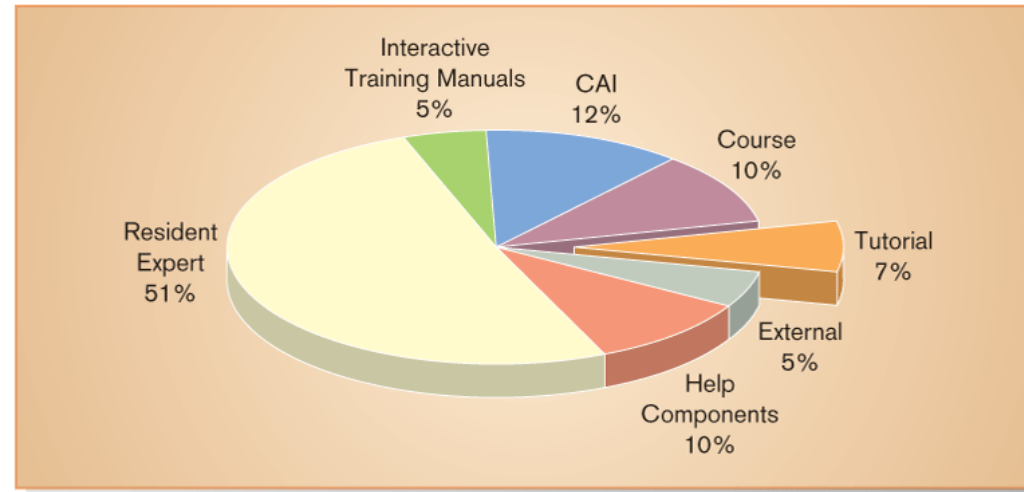
- Although the documentation is prepared to assist user throughout the system's life cycle, but it's important to train users with the new system.
- Normally, the team use the documentation prepared to assist them in training session.
- Converting from old to new system necessitates that the system users be trained and provided with documentation that guide them through the new system.
- Training can be one to one; however train a group of people is preferred.
- Group training can reduce the time and cost spend to conduct the training, and also encourage the group participates and feedback towards the system.
- There are few things that need to be considering during the training session.
- The content of the training session should be an introduction to the system, and system manual.
- Potential training topics
 - Use of the system
 - General computer concepts
 - Information system concepts
 - Organizational concepts
 - System management
 - System installation

Table 15-7 Seven Common Methods for Computer Training

1. Tutorial—one person taught at a time
2. Course—several people taught at a time
3. Computer-aided instruction
4. Interactive training manuals—combination of tutorials and computer-aided instruction
5. Resident expert
6. Software help components
7. External sources, such as vendors

(Source: Adapted from Nelson and Cheney, 1987.)

Figure 15-11 Frequency of use of computer training methods



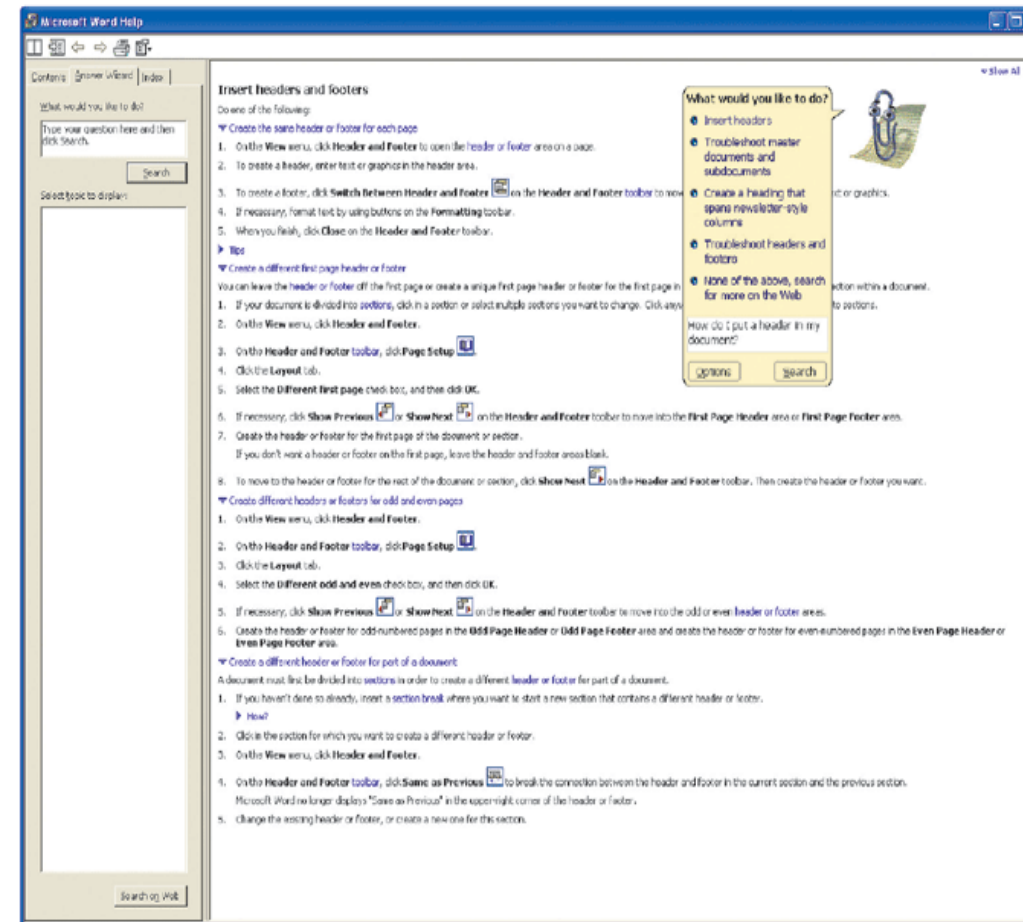
Source: From Nelson and Cheney, "Training End Users: An Exploratory Study," MIS Quarterly, 1987, used with permission.

By far the most common training method is informal, via interaction with an in-house expert on the software

Training methods

- Resident expert
- Computer-aided instruction
- Formal courses
- Software help components
- Tutorials
- Interactive training manuals
- External sources, such as vendors
- Electronic Performance Support System (EPSS)
- Component of a software package or application in which training and educational information is embedded

Figure 15-12 A Microsoft Office Assistant™ note



Electronic Performance Support Systems (EPSS), like Microsoft Office Assistant, are components of software applications that embed training and information for the user, in the form of tutorials, expert systems, and hyperlink jumps to reference topics.

- **Training Methods:**

- **Instructor-led training**

- It involves both trainers and trainees, who have to meet at the same time, but not necessarily at the same place. The training session could be one-on-one or collaborative. It is of two types –
 - **Virtual Classroom**
 - In this training, trainers must meet the trainees at the same time, but are not required to be at the same place. The primary tools used here are: video conferencing, text based Internet relay chat tools, or virtual reality packages, etc.
 - **Normal Classroom**
 - The trainers must meet the trainees at the same time and at the same place. Their primary tools used here are blackboard, overhead projectors, LCD projector, etc.

- **Self-Paced Training**

- It involves both trainers and trainees, who do not need to meet at the same place or at the same time. The trainees learn the skills themselves by accessing the courses at their own convenience. It is of two types –
 - **Multimedia Training**
 - In this training, courses are presented in multimedia format and stored on CD-ROM. It minimizes the cost in developing an in-house training course without assistance from external programmers.
 - **Web-based Training**
 - In this training, courses are often presented in hyper media format and developed to support internet and intranet. It provides just-in-time training for end users and allow organization to tailor training requirements.

Supporting Information Systems Users

- **Support:**

- Providing ongoing educational and problem solving assistance to information system users.
- For in-house developed systems, support materials and jobs will have to be prepared or designed as part of the implementation process.
- Support is extremely important to users
- Providing support can be expensive and time-consuming
- Support is extremely important to users
 - J.D. Power and Associates survey found user support to be number one criterion contributing to user satisfaction with personal computing
- One approach is through automation
 - Internet-based online support forums
 - On-demand fax
 - Voice response systems
 - Knowledge bases

- Most organizations provide support by two means
 - Information center
 - Help desk
- Information System support through information center for information system users
 - Install new hardware or software and set up user accounts
 - Consult with users writing programs in fourth-generation languages
 - Extract data from organizational databases onto personal computers
 - Answer basic on-demand questions
 - Provide a demonstration site for viewing hardware and software
 - Work with users to submit system change requests
 - Provides disaster recovery and backup
 - PC maintenance
 - Writing newsletters and offering other types of proactive information sharing
 - Setting up user groups
- Providing Support via Help Desk
 - A single point of contact for all user inquiries and problems about a particular information system or for all users in a particular department
 - Requires
 - Technical skills: extensive knowledge about how to use the system and typical problems that can be encountered
 - People skills: good listening and communication, dealing with complaints and frustrations

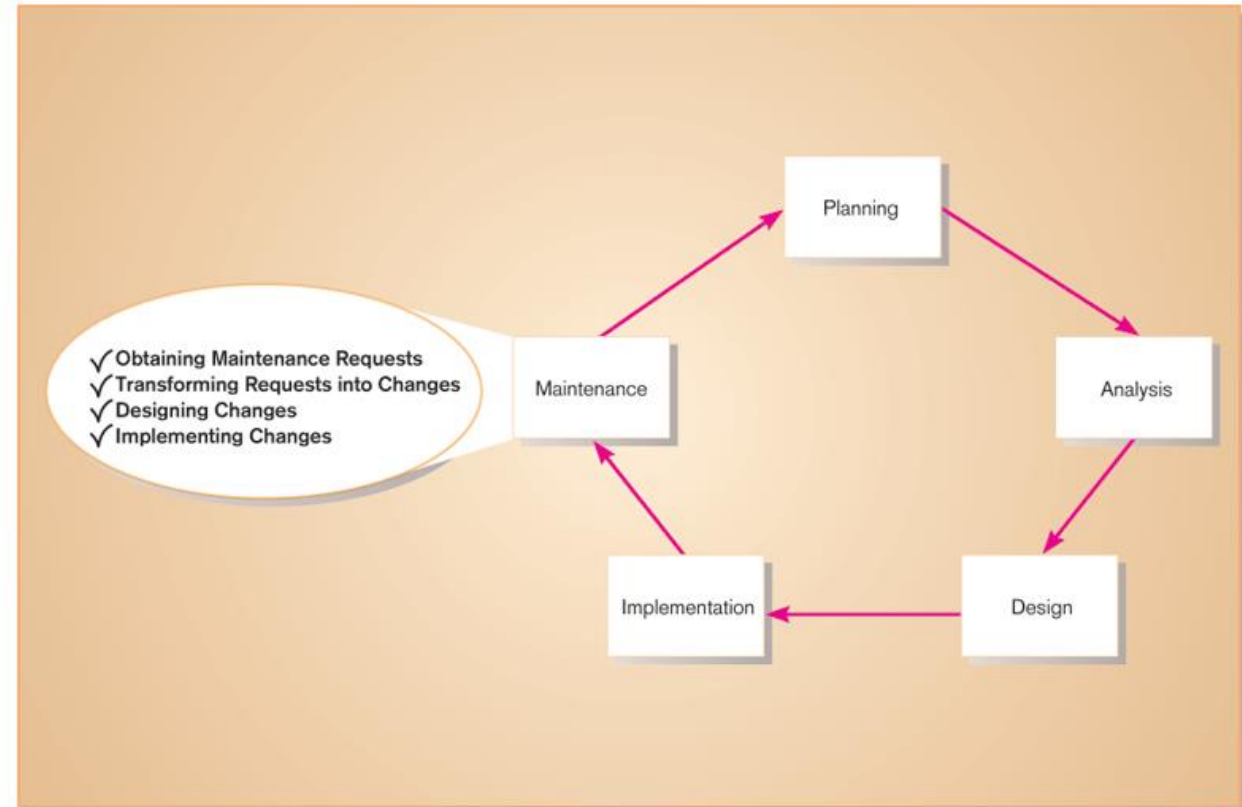
- **Classic implementation mistakes**
 - Using state-of-the-art technology
 - Increase time and cost estimates
 - Using low-cost personnel
 - Lack of code control
 - Use a source code library
 - Inadequate testing
- **Why Implementation Sometimes Fails**
 - Two conditions necessary for a successful implementation
 - Management support of the system under development
 - Involvement of users in the development process
 - Insights about implementation process
 - Risk
 - Commitment to the project
 - Commitment to change
 - Extent of project definition and planning
 - Realistic user expectations
 - Implementation success factors
 - Extent to which system is used
 - User's satisfaction with system

Ch-5.2 Maintaining Information System

Learning Objectives:

- ✓ Explain and contrast four types of system maintenance.
- ✓ Describe factors affecting maintenance costs.
- ✓ Describe maintenance management issues, including organizational structure, quality measurement, change management, and configuration management.

Figure 16-1 Systems development life cycle (SDLC)



Maintenance is like a mini-SDLC

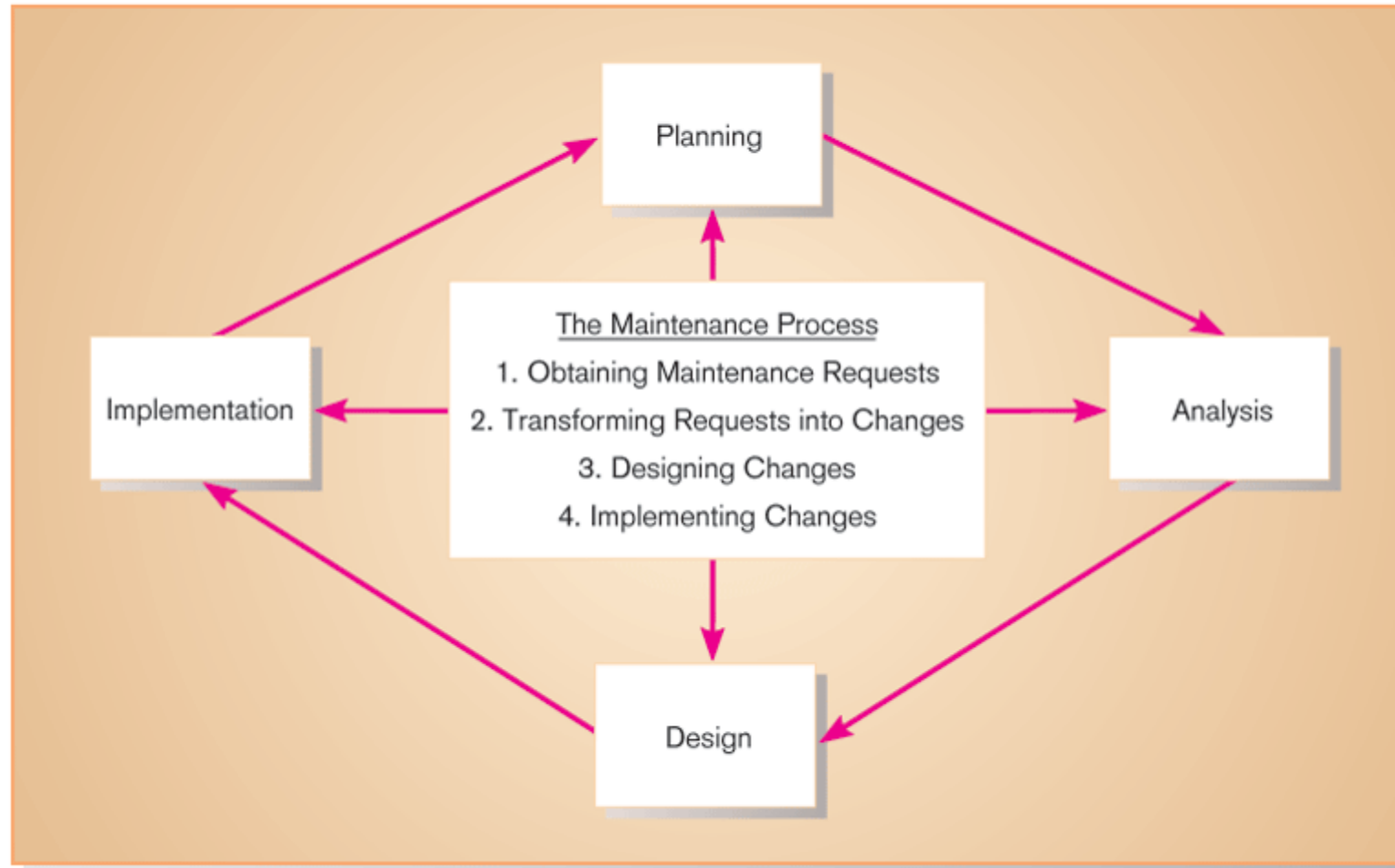
The Process of Maintaining Information Systems

- Process of returning to the beginning of the SDLC and repeating development steps focusing on system change until the change is implemented
- Maintenance is the longest phase in the SDLC
- Maintenance is the process of modifying a system after it has been delivered to the customer.
- The main purpose of maintenance is to modify and update system after delivery to correct faults and to improve performance.
- Changes made to a system to fix or enhance its functionality. Maintenance is a very broad activity that includes error corrections, enhancements of capabilities, deletion of obsolete capabilities, and optimization.
- The deliverables and outcomes from the process are the development of a new version of the system and new versions of all design documents created or modified during the maintenance effort.
- Four major activities:
 - Obtaining maintenance requests
 - Transforming requests into changes
 - Designing changes
 - Implementing changes

The Process of Maintaining Information Systems

- System modifications or maintenance is required to address the following reasons:
 - **Market Conditions** - Policies, which changes over the time, such as taxation and newly introduced constraints like, how to maintain bookkeeping, may trigger need for modification.
 - **Client Requirements** - Over the time, customer may ask for new features or functions in the system.
 - **Host Modifications** - If any of the hardware and/or platform (such as operating system) of the target host changes, software changes are needed to keep adaptability.
 - **Organization Changes** - If there is any business level change at client end, such as reduction of organization strength, acquiring another company, organization venturing into new business, need to modify in the original software may arise.
- Maintenance must be performed in order to:
 - Correct faults.
 - Improve the design.
 - Implement enhancements.
 - Interface with other systems.
 - Accommodate programs so that different hardware, software, system features, and telecommunications facilities can be used.
 - Migrate legacy software.
 - Retire software.

Figure 16-3 Maintenance activities parallel those of the SDLC.

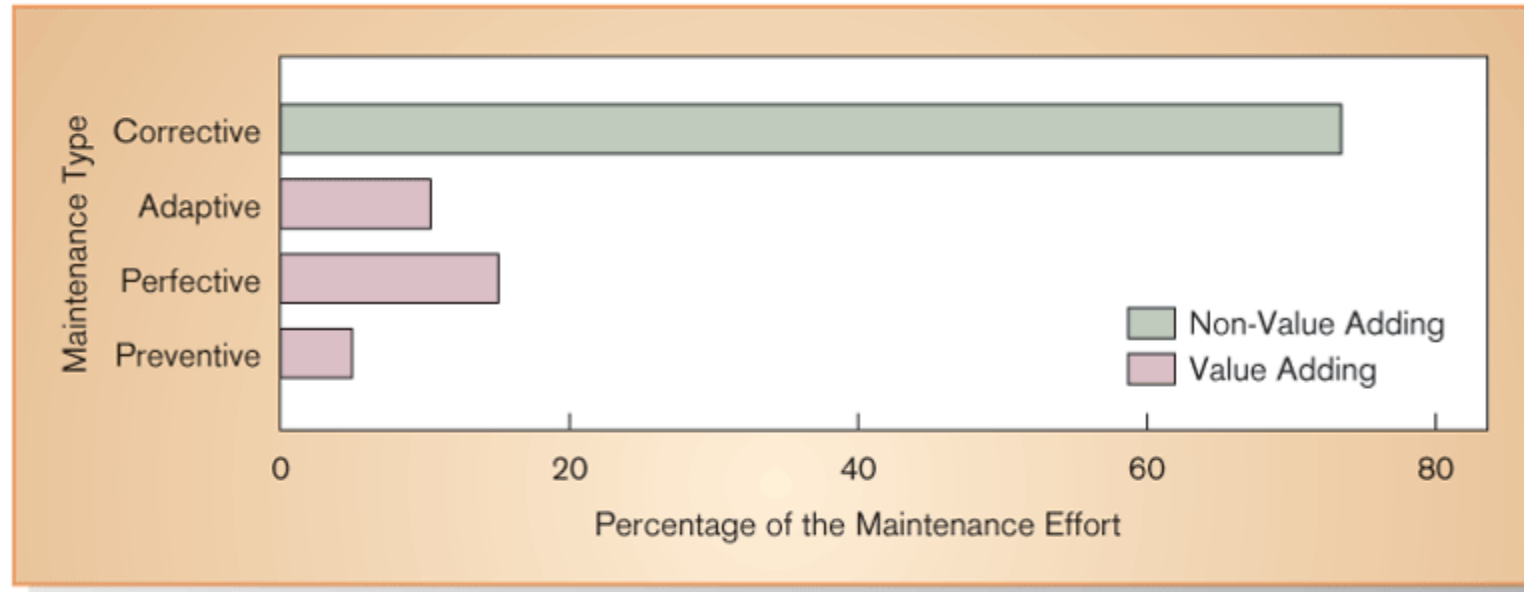


Maintenance is like a mini-SDLC

Types of System Maintenance

- **Corrective maintenance**
 - Changes made to a system to repair flaws in its design, coding, or implementation
 - Corrective maintenance of a system may be essential either to rectify some bugs observed while the system is in use, or to enhance the performance of the system.
- **Adaptive maintenance**
 - Changes made to a system to evolve its functionality to changing business needs or technologies
 - This includes modifications and updation when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware and software.
- **Perfective maintenance**
 - Changes made to a system to add new features or to improve performance
 - A system needs maintenance to support the new features that the users want or to change different types of functionalities of the system according to the customer demands.
- **Preventive maintenance**
 - Changes made to a system to avoid possible future problems
 - It goals to attend problems, which are not significant at this moment but may cause serious issues in future.

Figure 16-4 Types of maintenance (Adapted from Andrews and Leventhal, 1993)



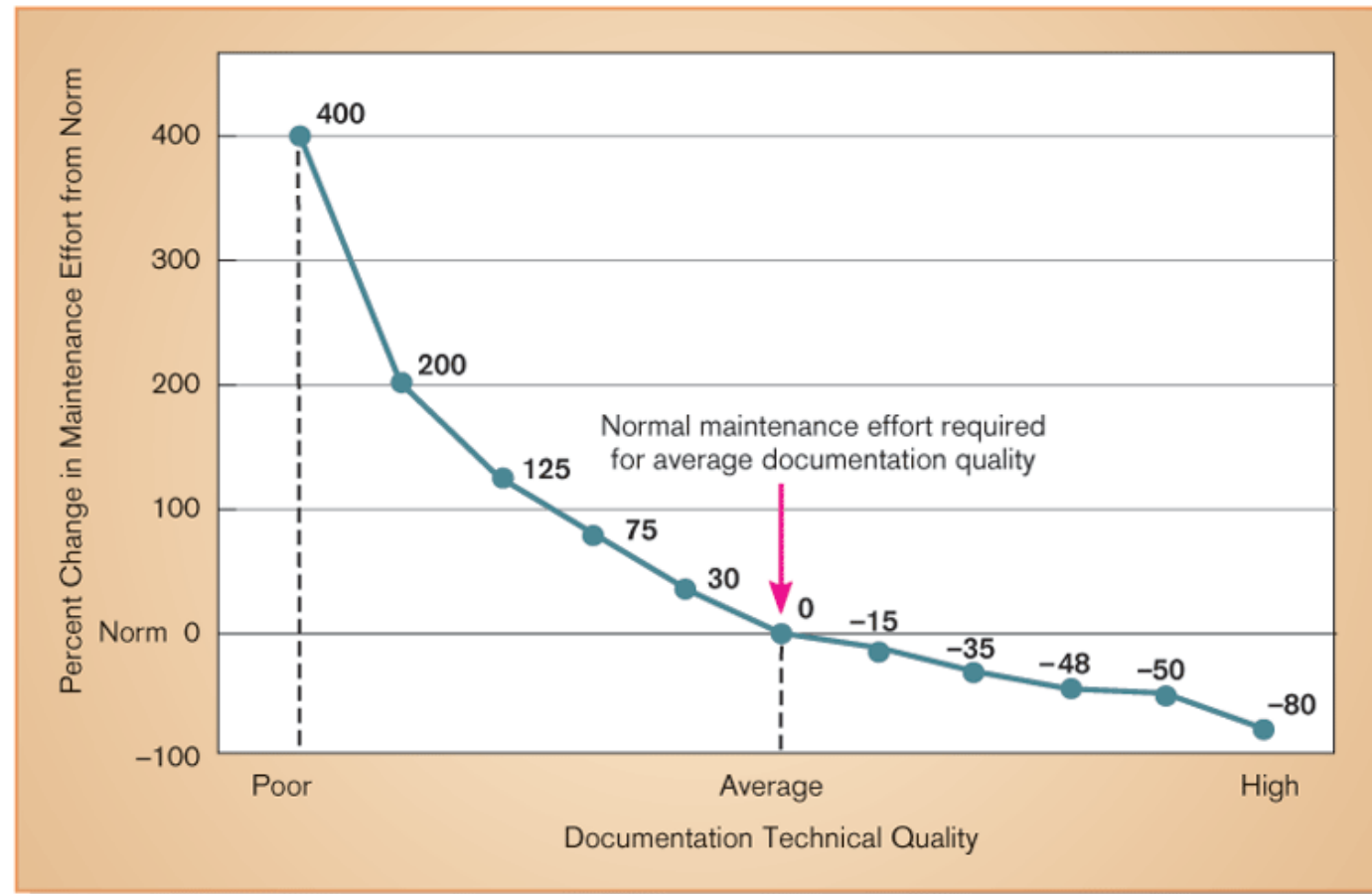
Source: Andrews D. C., and N. S. Leventhal. 1993. *Fusion: Integrating IE, CASE, JAD: A Handbook for Reengineering the Systems Organization*. Upper Saddle River, NJ: Prentice Hall.)

By far, most maintenance is corrective, and therefore urgent and non-value adding.

The Cost of Maintenance

- Many organizations allocate eighty percent of information systems budget to maintenance.
- Usually greater than development costs (2* to 100* depending on the application)
- Affected by both technical and non-technical factors
- Increases as software is maintained. Maintenance corrupts the software structure so makes further maintenance more difficult.
- Ageing software can have high support costs (e.g. old languages, compilers etc.)
- Various types of factors that influence system maintainability are:
 - Latent (hidden) defects.
 - Number of customers for a given system.
 - Quality of system documentation.
 - Maintenance personnel.
 - Tools.
 - Well structured programs.
- **Maintance Cost Factors:**
- Team stability
 - Maintenance costs are reduced if the same staff are involved with them for some time
- Contractual responsibility
 - The developers of a system may have no contractual responsibility for maintenance so there is no incentive to design for future change
- Staff skills
 - Maintenance staff are often inexperienced and have limited domain knowledge
- Program age and structure
 - As programs age, their structure is degraded and they become harder to understand and change

Figure 16-6 Quality documentation eases maintenance



Source: Hanna, M. 1992. "Using Documentation as a Life-Cycle Tool." *Software Magazine* (December): 41-46.

A well-documented system is easier to understand, and therefore easier to maintain.

Conducting System Maintenance

Managing Maintenance

- Number of people working in maintenance has surpassed number working in development.
- Three possible organizational structures:
 - Separate
 - Maintenance group consists of different personnel than development group.
 - Combined
 - Developers also maintain systems.
 - Functional
 - Maintenance personnel work within the functional business unit.

Advantages and Disadvantages

TABLE 14-2 Advantages and Disadvantages of Different Maintenance Organizational Structures

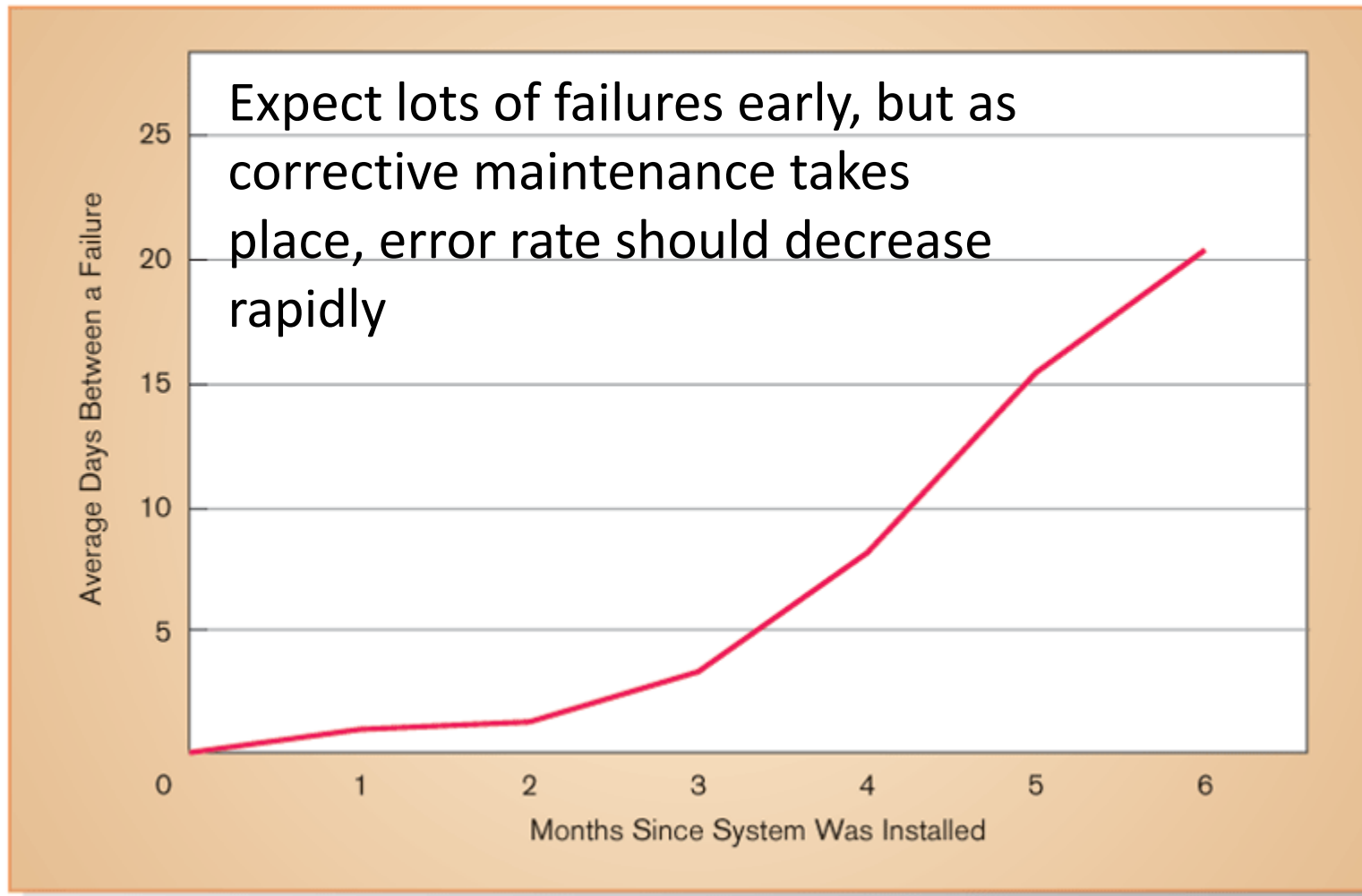
Type	Advantages	Disadvantages
Separate	Formal transfer of systems between groups improves the system and documentation quality	All things cannot be documented, so the maintenance group may not know critical information about the system
Combined	Maintenance group knows or has access to all assumptions and decisions behind the system's original design	Documentation and testing thoroughness may suffer due to a lack of a formal transfer of responsibility
Functional	Personnel have a vested interest in effectively maintaining the system and have a better understanding of functional requirements	Personnel may have limited job mobility and lack access to adequate human and technical resources

Conducting System Maintenance

Measures of Effectiveness

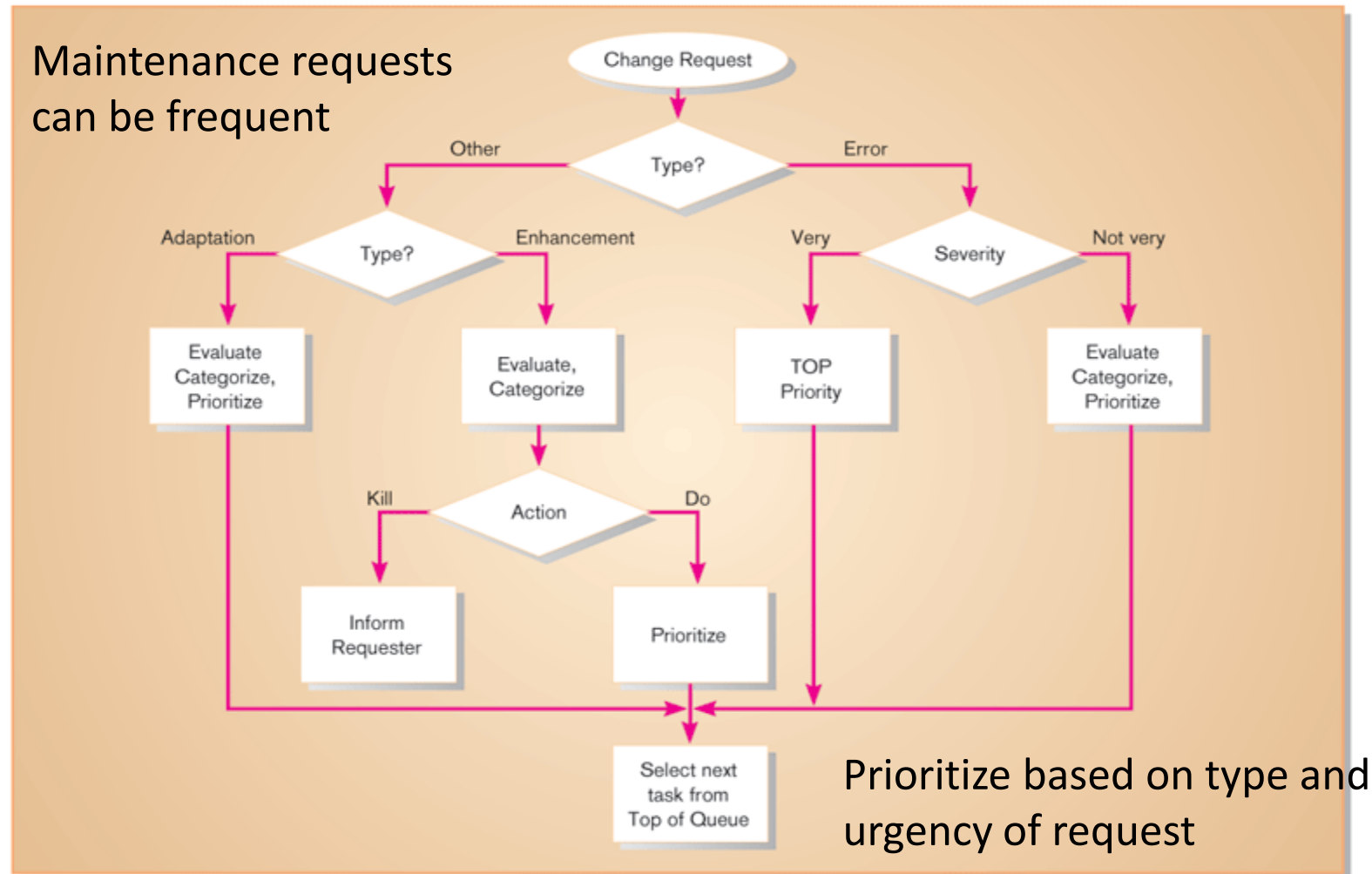
- Important factors to consider
 - Number of failures
 - Time between each failure
 - Type of failure
- Mean time between failures (MTBF)
 - A measurement of error occurrences that can be tracked over time to indicate the quality of a system

Figure 16-7 How the mean time between failures should change over time



Controlling Maintenance Requests

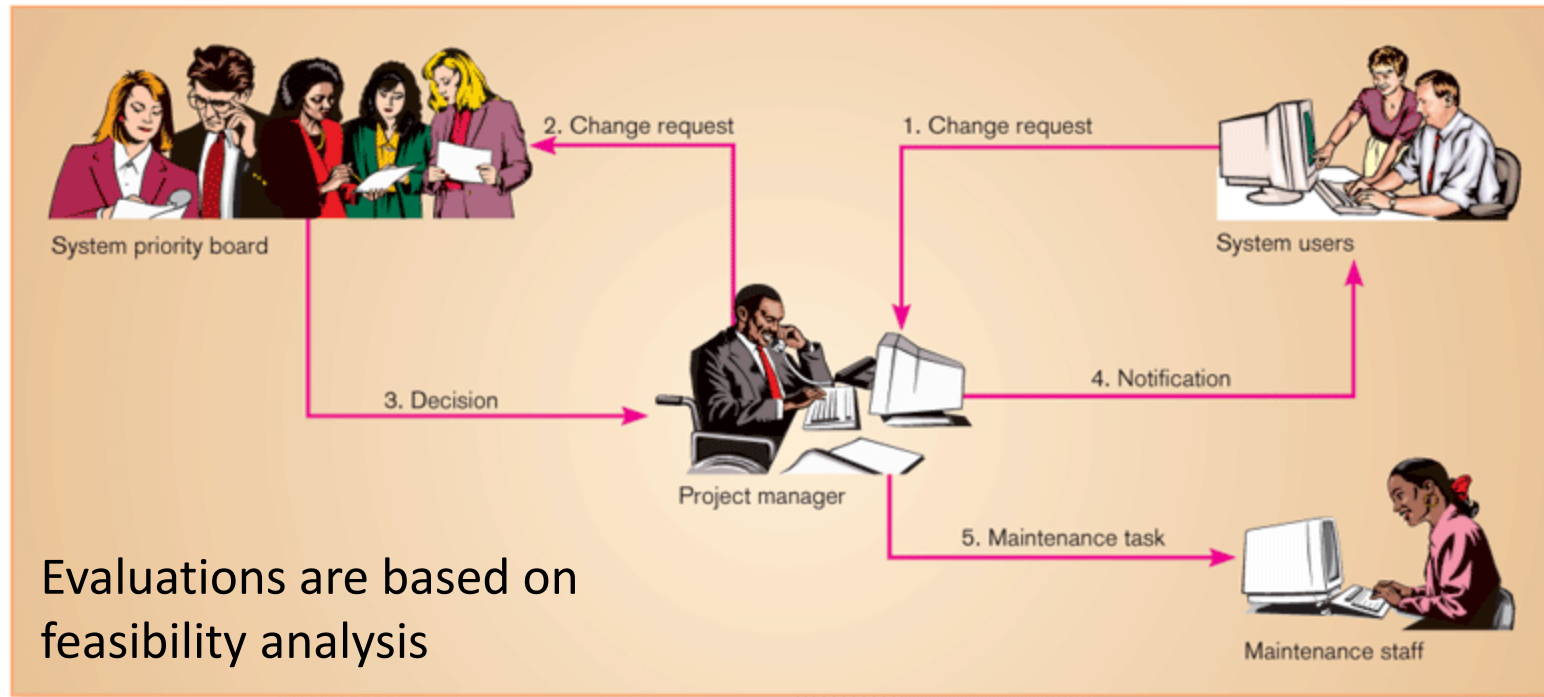
Figure 16-8 Flowchart of how to control maintenance requests



Source: Pressman, R. S. 2001. *Software Engineering: A Practitioner's Approach, 5th Edition*. New York: McGraw-Hill.

The Flow of a Maintenance Request

Figure 16-9 How a maintenance request moves through an organization



Configuration Management

- The process of assuring that only authorized changes are made to the system
- Baseline modules
 - Software modules that have been tested, documented, and approved to be included in the most recently created version of a system
- System librarian
 - A person responsible for controlling the checking out and checking in of baseline modules when a system is being developed or maintained
- Build routines
 - Guidelines that list the instructions to construct an executable system from the baseline source code

Configuration Management Tools

- Special software systems for managing system configuration
- Controls access to modules in the system library
- Two types
 - Revision control: modules labeled as frozen (unchangeable) or floating (checked out by programmer for modification)
 - Source code control: extend revision control to all interrelated modules
- Historical changes can be traced and previous versions can be reconstructed